

人像跟踪功能设计方案

1. 背景与目标

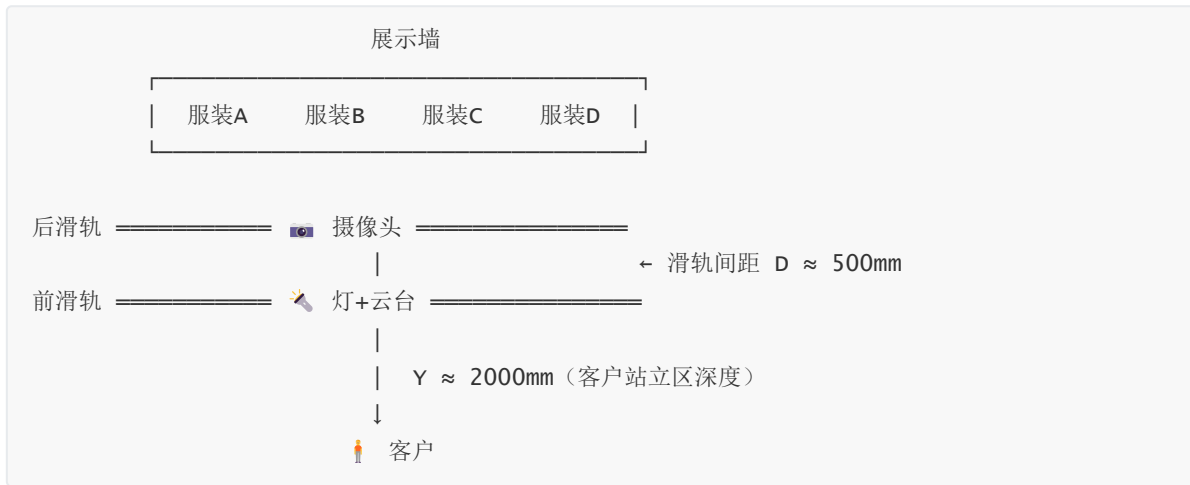
当前智慧服装店照明系统已实现：双色温 LED 照明、ToF 人体靠近检测、光照传感器自动调光、云台/滑轨远程控制。

本期目标：增加摄像头人像跟踪能力。当客户站在展示区，灯具自动跟随客户位置移动，让灯光始终打在客户身上。

关键约束：

- 图像采集 → 人体检测 → 坐标计算 → 全部在设备本地完成，不依赖云端
- 成本控制在 ¥250 以内
- 尽量复用现有硬件（ESP8266 + Nano 电机控制），不改 PCB

2. 物理布局



- 后滑轨**挂载摄像头，跟随客户水平移动
- 前滑轨**挂载云台 + 双色温灯，跟随客户水平移动
- 两条滑轨独立运动，由 ESP8266 → Nano 统一控制
- 客户站立深度假设为固定值 2m（可配置），简化测距问题

3. 硬件选型

核心方案

组件	型号	作用	价格	接口
摄像头	DFRobot HUSKYLENS 1	AI 人体检测 + 坐标输出	¥249	I2C
主控	ESP8266 (ESP-12E)	坐标换算 + 电机调度	已有	—

组件	型号	作用	价格	接口
电机驱动	Arduino Nano	滑轨/云台电机控制	已有	UART

为什么选 HUSKYLENS 1

对比项	HUSKYLENS 1	ESP32-CAM	Maix Dock K210
人体检测	✅ 出厂自带，按一下学习键	❌ 需自己训练/部署模型	⚠️ 需写 MaixPy 代码
坐标输出	✅ (x, y, w, h) 直接读	❌ 分类模型不输出坐标	✅ 需要开发
接线	4 根 (VCC/GND/SCL/SDA)	6+ 根	6+ 根
上手时间	5 分钟	数天	数小时
价格	¥249	~¥30	~¥130

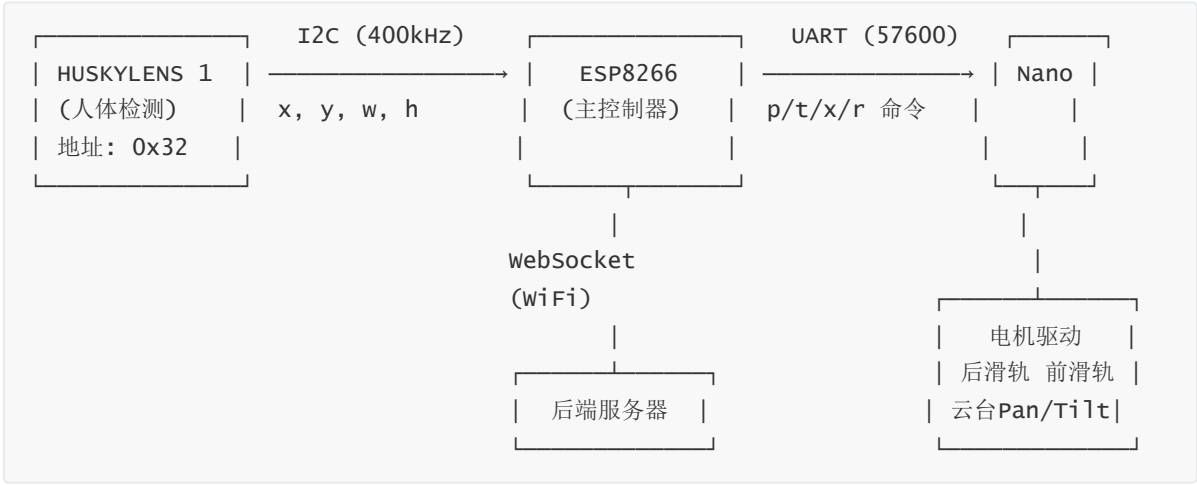
结论：HUSKYLENS 1 是唯一"即插即用"的选择。上电 → 切换到物体追踪模式 → 对着人按一下 learn 按钮 → 开始输出坐标。

购买链接

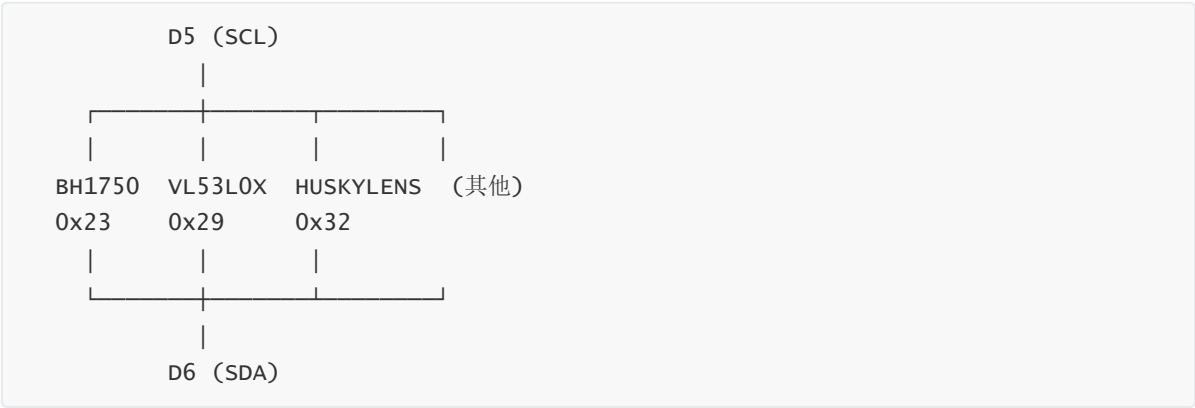
DFRobot 官方商城：<https://mall.dfrobot.com.cn/goods-2632.html>

4. 系统架构

整体数据流



I2C 总线设备



HUSKYLENS 的 I2C 地址 0x32 与现有的 BH1750 (0x23) 和 VL53L0X (0x29) 不冲突，直接共用 D5/D6 引脚。新增 0 根信号线。

I2C 总线冲突处理： `loop()` 中交替轮询——奇数帧跑 person tracking，偶数帧跑 ToF，每次只让一个设备占用 I2C 总线。

ESP8266 模块架构（代码层面）

```
src/
├─ main.cpp                # 主循环（trackTurn 交替调度）
├─ device/
│   ├─ person_tracker.cpp  # 【新增】跟踪核心逻辑
│   ├─ arm_controller.cpp  # 【修改】新增后滑轨控制
│   ├─ sensor_manager.cpp  # 【修改】初始化 HUSKYLENS
│   ├─ light_control.cpp   # 不变
│   └─ ota_manager.cpp     # 不变
├─ network/
│   └─ ws_client.cpp       # 【修改】新增 person_track 消息
├─ config/
│   └─ config_manager.cpp  # 【修改】新增跟踪参数持久化
└─ server/
    └─ local_server.cpp    # 不变
```

5. 坐标映射数学

5.1 已知量

符号	含义	默认值	来源
<code>x_pixel</code>	HUSKYLENS 输出检测框中心 X	0~320	每帧读取
<code>w</code>	画面宽度	320 px	固定
<code>y</code>	客户站立深度	2000 mm	可配置
<code>d</code>	前后滑轨间距	500 mm	实测后配置
<code>θ_h</code>	摄像头水平视场角	~60°	固定

5.2 步骤

Step 1 — 像素偏移 → 水平物理偏移

```
pct = (x_pixel - 160) / 160          // 归一化到 [-1, +1]
ΔX = pct × Y × tan(30°)              // 单位: mm
    = pct × Y × 0.577
```

Step 2 — 滑轨位置

```
后滑轨目标:  X_rear  = X_rear_current + step × sign(ΔX)
前滑轨目标:  X_front = X_rear + ΔX × (D / Y)    // 视差修正
```

前滑轨比后滑轨多一个视差修正项 D/Y ：因为摄像头在后方看到的人像位置，与灯光应该照射的位置，之间存在前后距离带来的视差。

Step 3 — 云台水平角度

```
pan = atan2(ΔX, Y) × 180 / π
pan = clamp(pan, -90°, +90°)
tilt = -10°    (固定, 照人体上半身)
```

5.3 数值示例

假设客户站在画面右侧 1/3 处 ($x_pixel = 213$ ，即 $pct = +0.33$)， $Y = 2000mm$ ， $D = 500mm$ ：

```
ΔX = 0.33 × 2000 × 0.577 = +381mm    // 客户偏右 38cm
后滑轨向右移动 1 step
前滑轨 = 后滑轨 + 381 × (500/2000) = 后滑轨 + 95mm
pan = atan2(381, 2000) × 57.3 = +10.8° // 云台向右转 11°
tilt = -10°
```

6. 防抖策略

摄像头检测存在抖动（相邻帧的检测框可能跳动几个像素）。如果不加防抖，滑轨和云台会不停微调。

机制	参数	效果
死区	中心 ±3% (~±10px)	人在画面中央附近不做任何移动
方向确认	连续 3 帧同一方向	过滤掉单帧噪声
丢失回中	人消失 > 3 秒	自动回到初始位置集中展示区
滑轨步进	每帧最多移动 rearStep mm	平滑过渡，不跳变

状态机



7. 通信协议

7.1 HUSKYLENS → ESP8266 (I2C)

使用 `HUSKYLENSArduino` 库（自动处理 I2C 包协议）。ESP8266 端代码：

```
#include <HUSKYLENS.h>

HUSKYLENS huskyLens;

// 初始化
huskyLens.begin(wire); // wire 已由 sensor_manager 配好 D5/D6

// 每帧读取
huskyLens.request(); // 请求一帧
while (huskyLens.available()) {
    HUSKYLENSResult r = huskyLens.read();
    // r.xCenter 0~320 检测框中心 X
    // r.yCenter 0~240 检测框中心 Y
    // r.width 检测框宽度（越大越近）
    // r.height 检测框高度
    // r.ID 目标 ID（0=无人）
}
```

7.2 ESP8266 → Nano (UART)

原有协议为单字符命令 + 参数 + 换行。本次新增 2 个命令：

命令	方向	参数示例	含义	备注
p	ESP→Nano	p 30\n	云台 Pan 设为 30°	已有
t	ESP→Nano	t -10\n	云台 Tilt 设为 -10°	已有
x	ESP→Nano	x 600\n	前滑轨移到 600mm	已有

命令	方向	参数示例	含义	备注
r	ESP→Nano	r 500\n	后滑轨移到 500mm	新增
R	ESP→Nano	R 10\n	后滑轨速度设为 10	新增

Nano 端需配合新增：接收到 r 命令时控制后滑轨步进电机。

7.3 ESP8266 ↔ 后端 (WebSocket)

后端 → ESP8266

```
// 启用/禁用跟踪
{"type": "person_track", "enabled": true}

// 调整跟踪参数（实时生效，自动保存到 Flash）
{
  "type": "person_track_config",
  "yPerson": 2000,
  "dRails": 500,
  "deadZone": 3
}
```

ESP8266 → 后端（状态上报，待实现）

```
{
  "type": "person_track_state",
  "tracking": true,
  "x_pct": 52.3,
  "pan": -12,
  "tilt": -8,
  "frontSlider": 580,
  "rearSlider": 520
}
```

8. 配置持久化

跟踪参数随 config.json 一起保存在 ESP8266 的 LittleFS 文件系统中，断电不丢失：

```
{
  "ssid": "NaHS",
  "password": "123456789",
  "serverHost": "device.genius.show",
  "httpPort": 80,
  "wsPort": 80,
  "personTrackEnabled": false,
  "yPerson": 2000,
  "dRails": 500,
  "deadZone": 3
}
```

首次启动全部使用默认值。通过 WebSocket 或后续开发本地 HTTP 接口可动态修改。